

```
# RECURSIVE CASE
for child in node['children']:
    preorderTraverse(child) # Traverse child nodes.
# BASE CASE
❷ return

preorderTraverse(root)
```

The equivalent JavaScript program is in *preorderTraversal.html*:

JavaScript

```
<script type="text/javascript">
root = {"data": "A", "children": [{"data": "B", "children":
[{"data": "D", "children": []}]}, {"data": "C", "children":
[{"data": "E", "children": [{"data": "G", "children": []},
{"data": "H", "children": []}]}], {"data": "F", "children": []}]}];

function preorderTraverse(node) {
    document.write(node["data"] + " "); // Access this node's data.
    ❶ if (node["children"].length > 0) {
        // RECURSIVE CASE
        for (let i = 0; i < node["children"].length; i++) {

            preorderTraverse(node["children"][i]); // Traverse child nodes.
        }
    }
    // BASE CASE
    ❷ return;
}

preorderTraverse(root);
</script>
```

The output of these programs is the node data in preorder order:

```
A B D C E G H F
```

When you look at the tree in [Figure 4-1](#), notice that preorder traversal order displays the data in left nodes before right nodes, and bottom nodes before top nodes.